Project Instructions

Team Formation

- Projects will be completed in **teams of 4 students**.
- Each team must select **one project** from the provided list.
- Carefully review the project description and deliverables before starting.

Submission Guidelines

- **Deadline:** Friday, May 8 at 11:59 PM.
- **Submission Platform:** GitHub Classroom.
- Your project code must be submitted along with **Docker files** so the program can be run as a **Docker container**.

Grading & Presentation

- Project is worth 20% of total course grade.
- After submission, additional instructions will be shared to schedule a **20-minute grading session** with the Teaching Assistants (TAs).
- **Presentation Requirements:**
  - Maximum duration: **20 minutes**.
  - You may use a **slideshow** to support your presentation.
  - You must **demonstrate your program live** on your laptop.
- **TA Evaluation:**
  - TAs will review your GitHub repository during the session.
  - You will be asked to run your program, provide input scenarios, and test edge cases.
  - TAs will ask questions to assess your team's understanding of the project.

Key Reminders

- Ensure your **Docker setup works correctly** so the program can be easily run as soon as you start your session. Technical issues with your Docker will deduct from the time your presentation time and will affect your grading.
- Keep your presentation concise and focused on demonstrating functionality.
- Be prepared to explain your design decisions, handle test cases, and answer technical questions.

📢 Important Note on Using LLMs

You are welcome to use **Large Language Models (LLMs)** to assist with your project.
However, you must follow these guidelines:

- **Validate all code**: Ensure that any code generated or suggested by an LLM is correct, functional, and fully understood by your team.
- **Take ownership of learning**: Use LLMs as a tool to support your work, not as a substitute for your own understanding.
- **Do not share your work with peers**: Sharing code between teams undermines the learning process. Each team must complete the project independently. Also, TAs have full access to your GitHub repositories and will be able to detect cases of plagiarism.

Remember: copying or sharing code doesn't just risk penalties—it **steals your peers' opportunity to learn** and weakens your own mastery of the material. **I trust you** to reasonably use LLMs and submit your own work.

Project 1: DDoS Mitigation Tool

TAs: Salma Kishk and Abdulrahman Abutur

Overview

You will build a **network monitoring system** that detects Distributed Denial of Service (DDoS) attacks through traffic analysis, establishes baseline behavior patterns, and automatically implements rate limiting or firewall rules to mitigate attack impact.

What are DDoS attacks? A DDoS attack employs multiple compromised systems (often part of a botnet) to flood a target—such as a website, server, or network—with massive amounts of traffic or requests. The goal is to overwhelm the target's resources (bandwidth, CPU, memory, or application capacity) so that legitimate users can no longer access the service.

Types of DDoS attacks:

- Volumetric attacks: Saturate bandwidth with huge amounts of traffic.
- Protocol attacks: Exploit weaknesses in network protocols (e.g., SYN floods).
- Application-layer attacks: Target specific services or applications with seemingly legitimate requests at abnormal rates.

Impact of DDoS attacks:

- Service disruption: Websites, applications, or APIs become unavailable.
- Financial loss: Downtime can cost businesses thousands to millions in lost revenue.

- Reputation damage: Customers lose trust if services are frequently unavailable.
- Operational strain: IT teams must divert resources to respond to the attack.

Why detecting DDoS is important:

- Early detection allows organizations to respond before the attack fully cripples services.
- Minimizes downtime by triggering automated defenses (e.g., rate limiting, filtering, rerouting).
- Protects users by ensuring legitimate traffic isn't blocked.
- Supports compliance with cybersecurity regulations and best practices.

Where DDoS mitigation tools are used:

- Cloud providers (AWS, Azure, Google Cloud) → protect hosted applications.
- Enterprise networks → safeguard internal systems and customer-facing services.
- ISPs and telecoms → defend infrastructure and prevent large-scale outages.
- Critical infrastructure (banks, hospitals, government services) → ensure essential services remain online.

This project teaches **network security**, **anomaly detection**, and demonstrates **defensive techniques** used by infrastructure providers and security operations centers.

Your Tool Must Work on PCAP Files with Two Modes

Your DDoS mitigation tool must accept **pcap files** as input. The tool must support **two distinct modes of operation**:

1. **Learning Mode:** The tool reads a pcap file containing normal traffic, extracts features, builds a baseline profile, and **saves it** to disk so it can be reused later without relearning.
2. **Detection Mode:** The tool loads a previously saved baseline, reads a pcap file containing traffic to be analyzed compares them against the baseline, and triggers detection + mitigation when anomalies are found.

*Note: If you use ML and If you cannot find any appropriate PCAP dataset to train your **ML model**, you can use CSV, but make sure your tool ingests PCAP in detection mode*

During grading You must come to the evaluation with a **pre-learned baseline** from normal traffic and a **corresponding attack pcap** ready to test on. We may or may not provide additional pcap files for testing.

Step-by-Step Instructions

1. Learn about DDoS attacks

Spend some time learning about DDoS attack types and how to detect them. For example,

Detection approaches:

- Volumetric → obvious traffic spikes
- Protocol → unusual packet patterns
- Application → suspicious request rates

---

2. Establish Traffic Baseline

- Obtain a dataset with DDoS data to build your tool for. You can use one of the publicly available datasets listed below or suggest another.
- Use a packet sniffing and/or traffic analysis tool to analyze the `pcap` files. A few are suggested below.
- Compute traffic statistics per time window, e.g., packet counts per protocol, source/destination IP distributions, port usage, bandwidth trends.
- Store baseline metrics to capture normal traffic patterns.
- Build statistical models (e.g., mean, std. deviation, percentiles, or machine learning, etc.) for anomaly detection.

---

3. Implement Anomaly Detection

- Compare against baseline patterns.
- Flag deviations (e.g., traffic exceeds baseline mean by X).
- Example indicators:
  - Sudden traffic spikes
  - Protocol distribution shifts
  - Source IP concentration
  - Port concentration
- Combine different indicators/signals into a composite anomaly score.
- Optimize performance, e.g., rolling windows, incremental aggregation, sampling.

---

4. Automated Mitigation

- **Rate limiting**: Token bucket algorithm.
- **Firewall rules**: Use `iptables` to block malicious sources.
- **Safeguards**:
  - Lenient rate limiting first
  - Gradual restrictions
  - Whitelist critical IPs

---

5. Alerting

- Notify admins via **email, Slack, WhatsApp, or other system notifications**.
- Include details: attack type, affected services, mitigation status, source IPs/countries.
- Implement **alert ranking** to avoid notification fatigue.

---

6. Monitoring Dashboards

- Show real-time traffic stats, attack indicators, active mitigations, historical trends.
- Example visualizations:
  - Bandwidth usage
  - Packet rates
  - Protocol distribution
  - Geographic attack sources
- Provide admin controls: adjust rate limits, whitelist/blacklist IPs, trigger mitigations.

Suggested Datasets

You can start with one of the following publicly available datasets, e.g.:

- DDoS evaluation dataset; CIC-DDoS2019 (Canadian Institute for Cybersecurity, labeled DDoS traffic)
- CAIDA DDoS Attack Dataset (Center for Applied Internet Data Analysis)

These datasets provide labeled attack vs. normal traffic, with which you can test your anomaly detection algorithms.

We are aware that the CIC-DDoS2019 dataset is very large and the original UNB servers can be unreliable. You are not restricted to the suggested datasets — you can use **any publicly available DDoS pcap dataset** to build and test your tool, or **generate your own traffic** using tools like `hping3`, `Scapy` scripts, or any other tool.

Deliverables

Submission Guidelines

- Your team must submit the **project code files** along with the **Docker files** to your GitHub Classroom team repository.
- Use the dataset to design your project, ensuring that part of the dataset is reserved for **testing and/or demonstration purposes**.

Dashboard Demonstration

During your presentation to the TA, you should showcase a **dashboard** that *simulates* real-time traffic monitoring using the dataset.

Dashboard Features:

- **Time navigation**: Include a time slider (or otherwise) that allows movement between periods of normal traffic and periods with DDoS attacks.
- **Alerts & anomaly detection**: When DDoS attacks are detected, the dashboard should:
  - Flag deviations and indicators
  - Display anomaly scores
  - Recommend mitigation actions (with specific rate-limiting and IP table rules)
- **Notifications**: Functionality should allow sending alerts to the TA (via email, WhatsApp, or other channels).

Required Functionality

Your project must include:

Baseline & Monitoring

- Establish baseline traffic patterns
- Implement traffic monitoring
- Build rolling windows and efficient aggregation

Detection & Algorithms

- Develop anomaly detection algorithms
- Define thresholds and composite anomaly scores
- Integrate detection logic with baseline monitoring

Mitigation & Firewall

- Implement automated rate limiting (e.g., token bucket)
- Generate and manage firewall rules (e.g., iptables integration)
- Safeguards: whitelist critical IPs, gradual restrictions

Dashboard & Alerts

- Build admin dashboard with time slider and visualizations
- Display alerts, anomaly scores, and mitigation actions
- Implement notification system (email, Slack, WhatsApp)
- Ensure smooth TA demonstration with interactive controls

Recommended Libraries & Tools

- **Scapy** / **PyShark** / **dpkt** → packet sniffing & traffic analysis
- **Pandas** / **NumPy** → statistical modeling & anomaly detection
- **Matplotlib** / **Seaborn** / **Plotly** → dashboards & visualization
- **iptables** / **ufw** → firewall rule automation
- **Flask** / **Django** / **Streamlit** / **Plotly dash** → web-based monitoring dashboard
- **Celery** / **APScheduler** → background monitoring & alerting tasks
- **PyWA** / **SMTP** → alert notifications

Rubric

The rubric below outlines the **high-level expectations** for the project.
A more detailed rubric — including **individual and team grading criteria** — will be shared with you soon.

Note that the required functionality has been split into **four subprojects**, each for you to assign to a different team member.

- **Individual grading** will be based on each member's contribution to their subproject.
- **Collective team grading** will reflect the team's ability to integrate all parts into a cohesive final product.
- Effective teamwork is essential to complete the project successfully. You must manage your time as a team by:
  - Defining clear **task breakdowns**.
  - Setting **timely milestones**.
  - Performing **risk assessments** to anticipate and mitigate delays.
- Use your **GitHub Classroom repository** for:
  - Proper versioning of code and Docker files.
  - Collaboration through branches and pull requests.
  - Forking and reviewing project updates.
  - Maintaining a transparent history of contributions.

Levels

- **Level 3 (Excellent)** → Comprehensive, accurate, and professional execution
- **Level 2 (Satisfactory)** → Functional but with gaps in depth, clarity, or polish
- **Level 1 (Needs Improvement)** → Incomplete, unclear, or lacking core functionality

---

Criteria Table

Criterion	Level 3 (Excellent)	Level 2 (Satisfactory)	Level 1 (Needs Improvement)	Mapped Sub-Project
Demonstration & Presentation	Clear, well-structured demo; dashboard intuitive; within time limit	Demo shows basic functionality but lacks polish; dashboard limited	Demo confusing/incomplete; dashboard missing; presentation disorganized	Dashboard & Alerts
Answering TA Questions	Confident, precise explanations; deep understanding of design and algorithms	Answers most questions but limited detail; partial understanding	Struggles to answer; little understanding of code/concepts	Detection & Algorithms
Technical Detail & Functionality	Code well-structured, documented, runs in Docker; all features implemented	Code runs with minor bugs; most features implemented	Code fails to run; Docker missing; major features absent	Baseline & Monitoring / Mitigation
Interactivity & Dashboard	Interactive dashboard, anomaly visualization, admin controls	Dashboard shows traffic/alerts but limited interactivity	Dashboard static or missing; no interactivity	Dashboard & Alerts
Anomaly Detection Performance	Accurate detection; composite scores meaningful; false positives minimized	Detection works but simplistic; thresholds hard-coded; moderate errors	Detection unreliable/missing; anomalies not flagged	Detection & Algorithms
Mitigation Strategy	Automated rate limiting + firewall rules; safeguards included; clear recommendations	Basic mitigation (rate limiting or firewall); limited safeguards	Mitigation missing/ineffective; no safeguards	Mitigation & Firewall
Alerting & Notifications	Alerts; detailed anomaly info + mitigation actions	Alerts functional but missing details	Alerts missing or non-functional	Dashboard & Alerts

🆕 NEW: Clarifying Instructions and Grading Details

> Project worth **20 % of the course grade**. Each criterion is graded on a 4-level scale (L0 / L1 / L2 / L3) and weighted by a weight **W**. Per-sub-project score = `Σ (level × W) / Σ (3 × W)`.

---

1. Day-of-Grading Plan

The total demonstration (demo) time is **20 minutes**.

The core idea of this project is to design a dashboard that visualizes the evolution of network traffic captured in a `pcap` file over time. The dashboard should enable users to:

- **Monitor traffic dynamics**: Display how traffic patterns change as time progresses.
- **Highlight anomalies**: Identify and visually emphasize unusual or suspicious traffic behavior.
- **Generate alerts and notifications**: Provide timely warnings when anomalous activity is detected.
- **Suggest mitigating actions**: Offer guidance to users on possible responses to detected anomalies.

This is a **design-focused project**, you have the freedom to decide:

- How traffic is best visualized (e.g., timelines, heatmaps, interactive graphs, etc.).
- How anomalies are highlighted (e.g., color coding, overlays, statistical summaries, etc.).
- What interactive functionalities to provide (e.g., filtering, drill-down views, exporting reports, etc.).

Your goal is to create an intuitive, informative, and actionable dashboard that helps users understand traffic behavior and respond effectively to anomalies.

Keep your answers during the demo concise. Prepare thoroughly beforehand — technical issues with Docker or your tool will eat into your demo time and may affect grading.

1. **Bring** your team's laptop(s) with the project ready to run.
2. **Arrive with**:
   - Pre-trained baseline **saved to disk** and loadable. *Note that this baseline may or may not be a ML model; if it is, make sure it is trained before the demo starts.*
   - Working `docker compose up` (or equivalent one-liner) to start your dashboard.
   - Latest version of your code pushed to GitHub *before* the session.
   - You will need to show your dashboard ingesting a `pcap` file, processing it during the demo, and showcasing its capabilities. Note that you can use `csv` files during project development as you see best but your tool needs to ingest a raw `pcap` file.
   - Come ready with a test pcap that you did not use during development. Your client (TA) will have 2 reserved pcap files — one containing normal traffic and one containing an attack — that they may

ask you to validate your tool against. *Note that the normal traffic pcap alone will not be sufficient to train a ML model, so you must arrive with your model already trained and the corresponding attack pcap ready for demonstration.*

3. **You will be asked to**:
   - Boot your tool with the baseline loaded.
   - Run your monitoring tool on the `pcap` and walk through the dashboard live.
   - Show: anomaly detection/scores, alerts firing with details, mitigation rules generated, a notification arriving at a TA-supplied address.
   - Walk through your tool's deliverables: dashboard, generated firewall rules, sent notification.
4. **You may be asked to**:
   - Highlight any work that goes **beyond** the base requirements (extra detectors, novel features, performance optimisations) — this is considered during grading.
   - Explain portions of your code on GitHub.
   - Highlight each member's tasks, roles, and contributions.
5. **It is your team's responsibility** to demonstrate that every requirement is functioning correctly. **Failure to demonstrate a feature results in deduction for that criterion**, even if the feature exists in the code.

---

2. Levels & Scoring

Level	Meaning	Points
L3	Excellent — comprehensive, accurate, professional	3 × W
L2	Satisfactory — works but with gaps in depth, polish, or coverage	2 × W
L1	Needs Improvement — attempted but broken or unreliable	1 × W
L0	Not Implemented — feature entirely absent	0 × W

> **L1 vs L0:** L1 = they tried, the code path exists, but it doesn't work right. L0 = the feature simply isn't there.

Per-sub-project score = `Σ (level × W) / Σ (3 × W)` × 100 %.

Per-student final grade:

- Each student should ideally own one sub-project (A, B, C, or D). They get their sub-project's score as their *individual* portion. In case this is not the case - no problem, the client/TA will discuss with you to

understand how the work was distributed to grade you appropriately. *In this case, the client will be allowed some freedom in subjectively distributing the grades in the way they deem fair.*

- The portion of each student's grade is based on work **shared by all 4 members equally**.
- Final = `0.7 × individual_sub-project + 0.3 × team_T_score`.
- Convert to course grade: `final % × 20 / 100` = points out of 20.

---

3. Team grade — applies to all 4 members equally

#	Criterion	L3	L2	L1	W
T.1	**Demonstration & flow**	Demo follows a clear script: raw PCAP → detection → notification → mitigation rule	Demo works but incoherent / skips steps	Disorganised, missing key steps	3
T.2	**TA Q&A**	Each member understands the entire project, all their teammate's parts conceptually, and can defend their sub-project's design and edge cases	Most questions answered; some shallow	Members can't explain their own code	3
T.3	**Docker reproducibility**	Container starts cleanly on a fresh laptop in < 2 min	Works after one manual step	Fails — eats into demo time	2
T.4	**Code quality & repo hygiene**	Sensible commits, clear README, no committed secrets, files organised	Works but messy commits / no README	Single commit / no docs / secrets in repo	1
T.5	**Architecture**	Member can explain/draw-out the functionality/architecture of their tool on paper / whiteboard, including data flow, modules, data persistence, etc.	Incoherent description	Can't explain the architecture	1

---

4. Sub-Project A — Baseline & Monitoring

#	Criterion	L3	L2	L1	L0	W

A.1	**PCAP ingestion**	Accepts arbitrary `.pcap` at runtime via dashboard upload **or** CLI; parsing succeeds without crash on the supplied file	Accepts only one input path; minor parse warnings on edge protocols	Hardcoded path or fails to parse		2
A.2	**Feature extraction**	Several meaningful features are extracted from the traffic to guide detection	A reasonable number of features are extracted, mostly correct and relevant	Very few features are extracted, or only one overall feature is used	---	3
A.3	**Baseline persistence**	Saved to disk in a versioned format with relevant metadata; reloadable across runs	Saved + reloadable but missing metadata	Reload is broken	No persistence at all	3
A.5	**Detection model depth**	Insighful and correct application of statistical measures or correct training and validation of ML model	Incorrect, faulty, or unjustified application of statistical measures or ML model	Hardcoded thresholds, no actual baseline math		3

> *Footnote 1* — During evaluation, the team must be able to explain: how their model detects network anomalies (e.g., what window (time) size does your model use for the network traffic, if any? what statistics does your model compute for the windows, if any? what features in the raw data, or generated from the data, is used by the model to detect anomalies? etc.)

Maps to: Baseline & Monitoring sub-project member.

---

5. Sub-Project B — Detection & Algorithms

#	Criterion	L3	L2	L1	W

B.1	**Speed/performance**	Tool runs in *near real-time* on the test PCAP	Slower than real-time but completes within demo budget	Stalls or never produces output	3
B.2	**Anomaly score**	Continuous score per window, not a boolean. Score graph rendered neatly and clearly alongside dashboard graphs	Score exists but not well-integrated into the dashboard, e.g., in a seperate log	Boolean output only	3
B.3	**Detection justification** *(see footnote 2)*	Each trigger reports why it was triggered, e.g., trigger metric, observed value, threshold, timestamp	---	Generic "attack detected" with no context	3
B.4	**Detection logic**	Detects all traffic anomalies; distinguishes one continuous attack from multiple separate attacks	Cannot distinguish seperate attacks	Misses obvious anomalies	3
B.5	**De-duplication**	Multiple triggers attributed to the same anomaly are collapsed to a single alert	---	Multiple alerts generated for a single anomaly (this can overwhelm the user)	1

> **Footnote 2** — You may decide to also print detections to the terminal as a fallback in case the dashboard fails to display them during demo.

Maps to: Detection & Algorithms sub-project member.

---

6. Sub-Project C — Mitigation & Firewall

#	Criterion	L3	L2	L1	W

C.1	**Rate limiting**	Generate meaningful/clear logic for rate limiting	Some rate limiter present but algorithm unclear or untestable	Missing (L0)	3
C.2	**Firewall rules**	Generates targeted `iptables` (or equivalent) rules with concrete IPs / ports during attack; rules visible in the dashboard mitigation panel	Generic rules that do not map to the detected attacks	Missing	3
C.3	**Whitelist (trusted IPs)**	Whitelisted IPs are never rate-limited or blocked even when their traffic crosses thresholds; whitelist editable from the dashboard	Whitelist exists in config but no UI to edit	Missing	2
C.4	**Blacklist (banned IPs)**	Blacklist editable from the dashboard; packet drops for blacklist visible in IP table	Blacklist exists in config only	Missing	1
C.5	**Gradual / safeguarded mitigation**	Progressive, clear, and effective logic, e.g., Lenient first (rate-limit) → progressive (block specific port) → severe (block IP); confidence-gated	Single-action response (always block)	No safeguards (one-shot block on first signal — risk of self-DoS)	2

Maps to: Mitigation & Firewall sub-project member.

---

7. Sub-Project D — Dashboard & Alerts

7.1 Dashboard

#	Criterion	L3	L2	L1	L0	W

D.1	**Graphs**	Multiple (>=4) graphs effectively showing disinct network statistics, e.g., packets/bytes vs time, unique source IPs over time, network graph connecting devices with links weighed by traffic volume. Synchronised with real timestamps (HH:MM:SS). Interactive, e.g., allows filtering, zoom, etc.	---	Few graphs, graphs uninformative, or static images		3
D.2	**Time slider**	Both ends independently draggable; min = first packet timestamp, max = last; charts update on drag	---	No range control		3
D.3	**Anomaly-score visualisation**	Score graph during detection mode with threshold line and highlighted alert windows	---	Absent		3
D.4	**Whitelist / Blacklist UI**	TA can add / remove IPs from both lists at runtime via the dashboard	View-only or one of the two	Neither		2
D.5	**Anomaly drill-down**	Clicking an alert reveals relevant, insightful information about the alert, e.g., timestamp, anomalous metric, real value vs baseline/threshold, contributing source IPs/flow/etc.	Static alert list with timestamps only	Single banner with no per-event detail		2
D.6	**Interactivity & polish**	Smooth: no layout shifts, charts redraw cleanly, no console errors	Works but jitters / has layout bugs	Crashes / blank states / console exceptions		1

7.2 Notifications

#	Criterion	L3	L2	L1	W
D.7	**Live channel delivery**	At least one of {email, Slack, WhatsApp, SMS} delivers a message **live during the eval** to a TA-supplied address	Channel configured and code path runs but no delivery to the TA's address	Channel missing or non-functional	3

D.8	**Notification content**	Includes insightful alert information	---	Generic "attack detected"	3
D.9	**Cool-down / fatigue control**	First alert sent immediately; subsequent identical alerts suppressed for a configurable cool-down (~1–2 min); a final summary on attack end	Cool-down exists but fixed at 0 (every event sent) or absurdly long	No cool-down — fires every window	2

Maps to: Dashboard & Alerts sub-project member.

---

8. Sub-Project Weight Summary

Sub-project	Σ W	Max points
Shared (T)	10	30
Baseline & Monitoring (A)	11	33
Detection & Algorithms (B)	13	39
Mitigation & Firewall (C)	11	33
Dashboard (D)	14	42
Notifications (D)	8	24